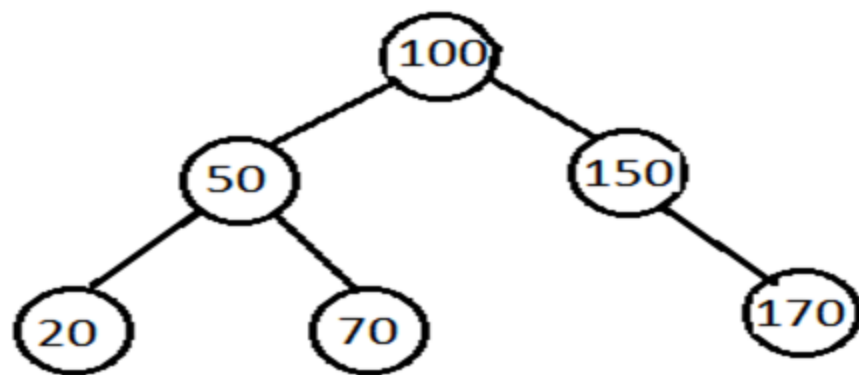




Binary Search Tree Data Structure

- Binary Search Tree.
- Insert node in the binary search tree
- Element of Tree.
- Binary Tree.
- Benefits of Binary Tree.
- Types Of Binary Tree.
- Binary Tree Representation.

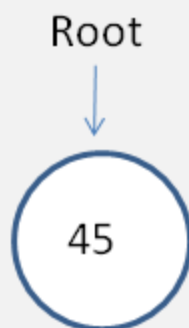
- A binary search tree follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.
- Searching an element in the Binary search tree is easy as we always have a hint that which subtree has the desired element.
- As compared to array and linked lists, insertion and deletion operations are faster in BST.



Create BST with data - 45, 15, 79, 90, 10, 55, 12, 20, 50

- First, we have to insert 45 into the tree as the root of the tree.
- Then, read the next element; if it is smaller than the root node, insert it as the root of the left subtree, and move to the next element.
- Otherwise, if the element is larger than the root node, then insert it as the root of the right subtree.

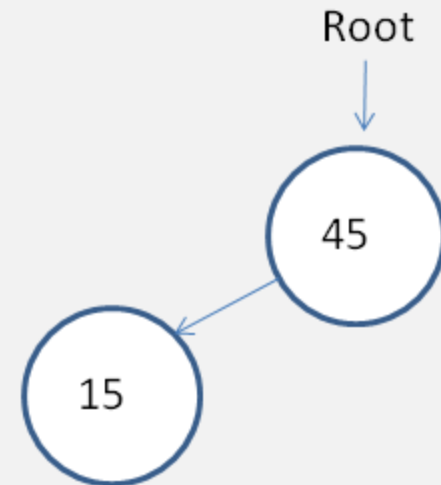
- Step 1.
Create Root



Inserting Node in Binary Search Tree

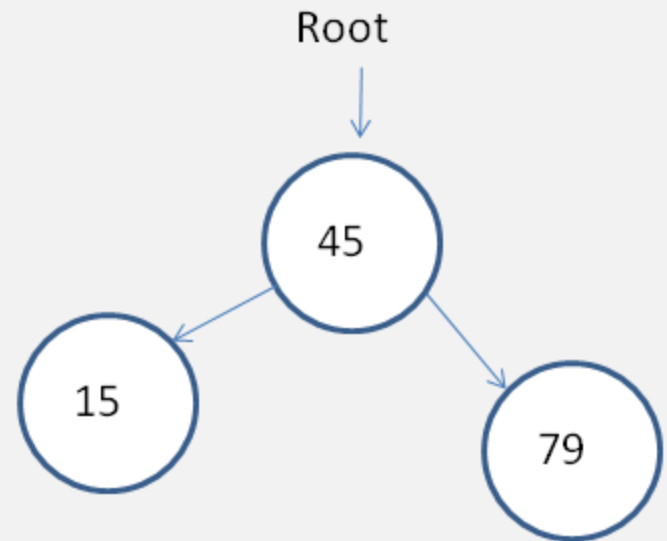
Step-2

Insert 15. As 15 is smaller than 45, so insert it as the root node of the left subtree.



Step 3 - Insert 79.

As 79 is greater than 45, so insert it as the root node of the right subtree.



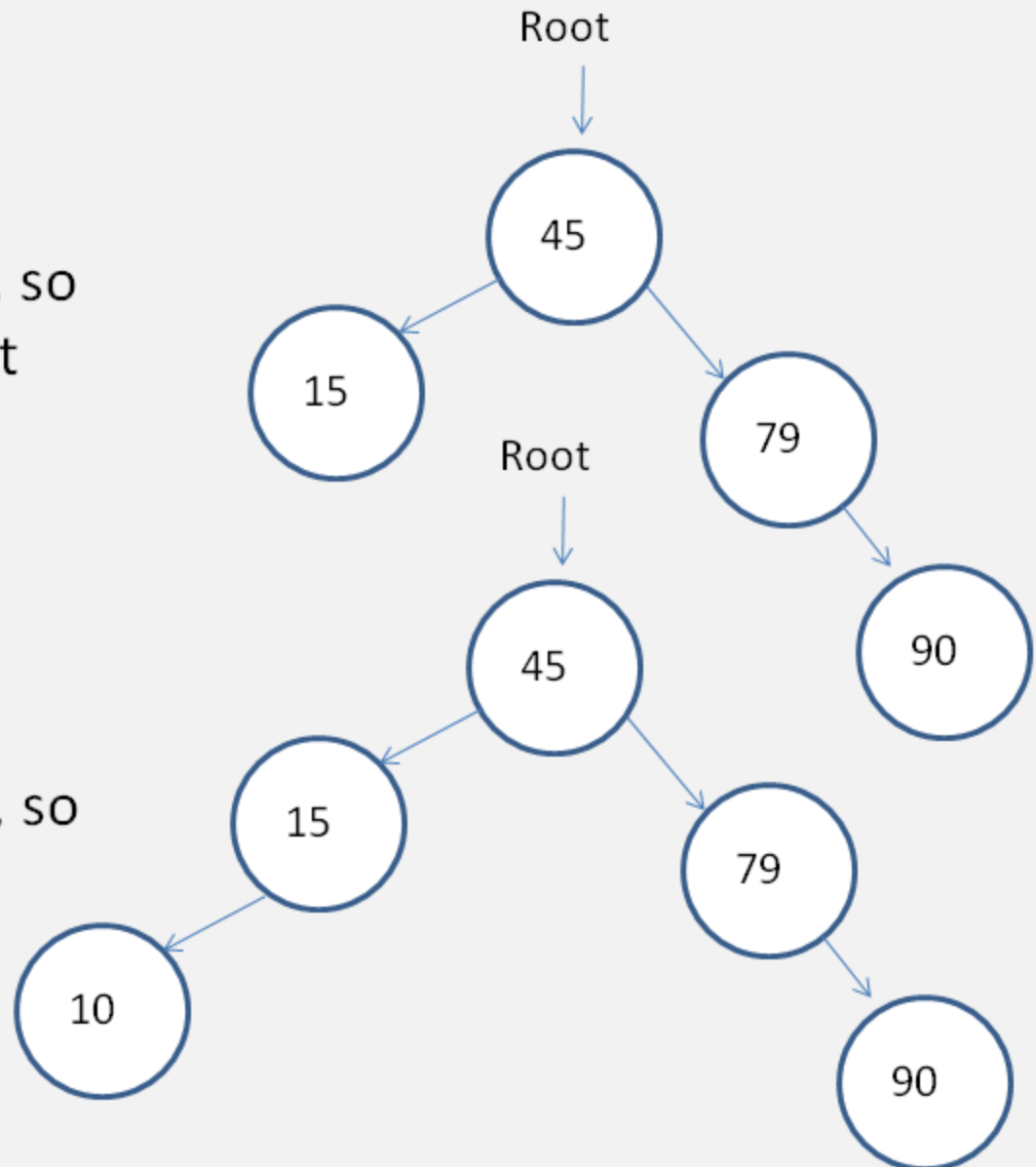
Inserting Node in Binary Search Tree

Step 4 - Insert 90.

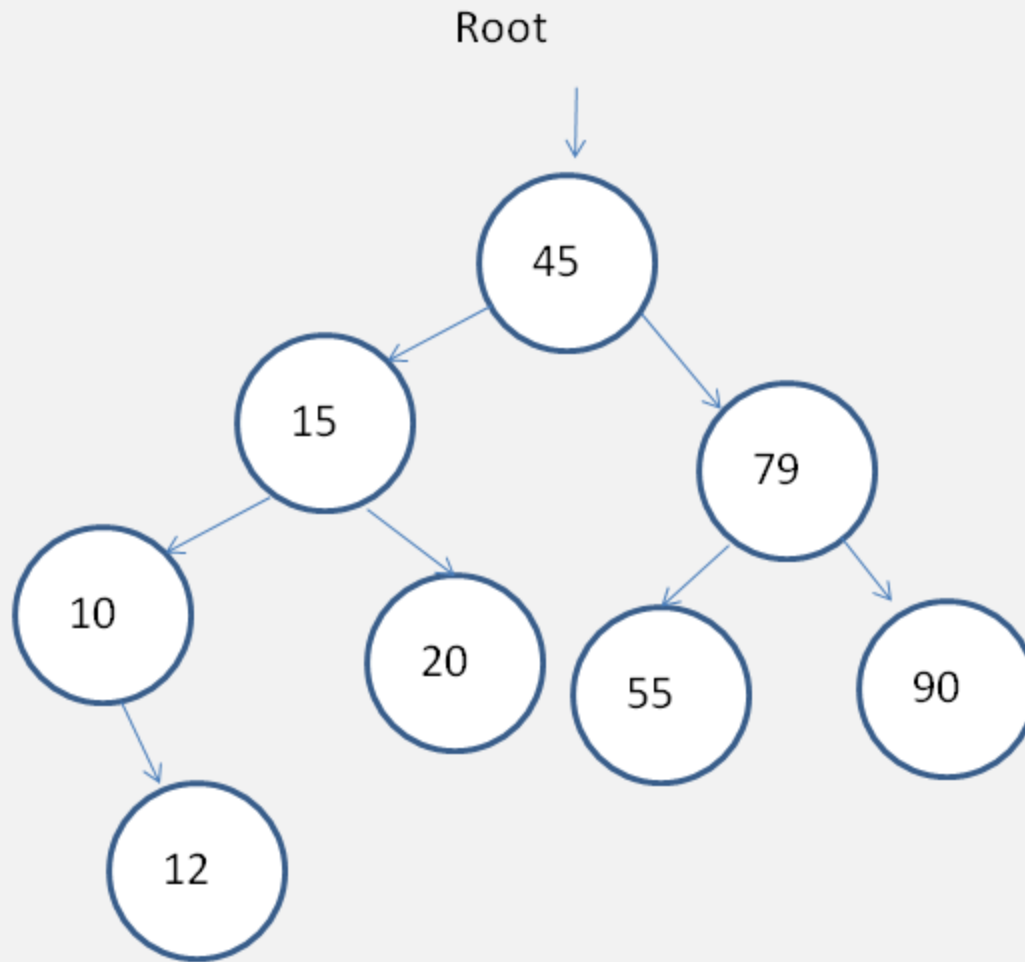
90 is greater than 45 and 79, so it will be inserted as the right subtree of 79.

Step 5 - Insert 10.

10 is smaller than 45 and 15, so it will be inserted as a left subtree of 15.



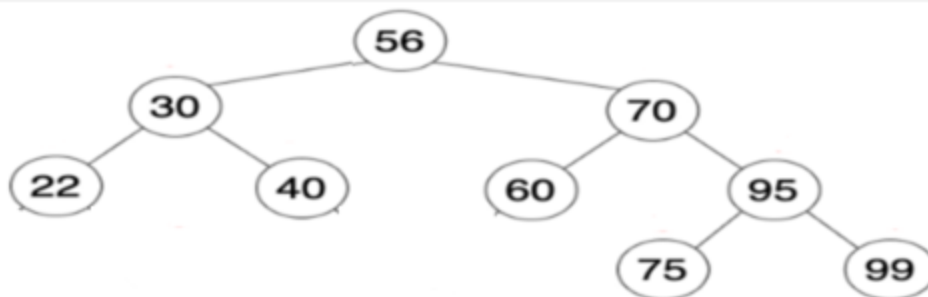
Final Binary Search Tree



1. Insert Node
2. Delete Node
3. Search Node
4. Traversal of Tree
 1. Inorder
 2. Preorder
 3. Postorder

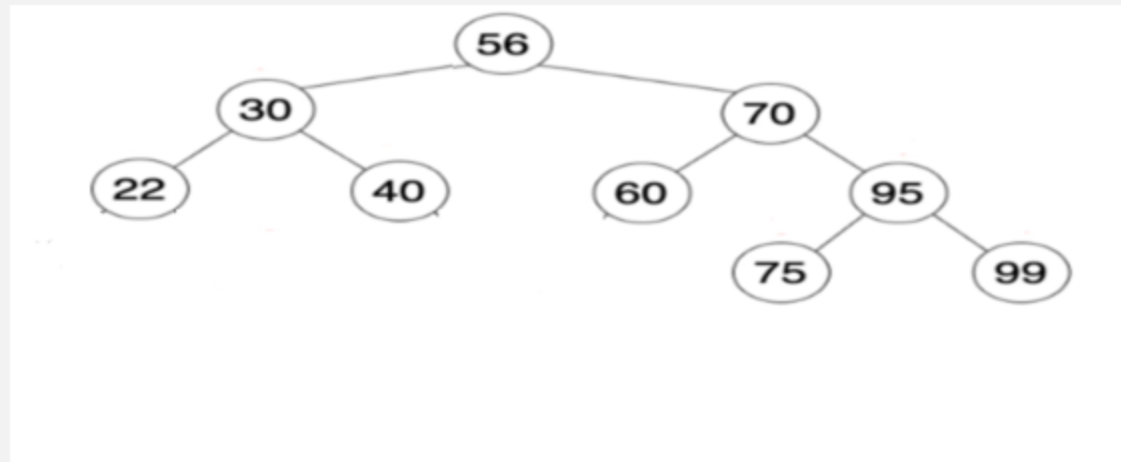
Inorder Binary Search Tree

- 1) Create an empty stack S.
- 2) Initialize current node as root
- 3) Push the current node to S and set current = current->left until current is NULL
- 4) If current is NULL and stack is not empty then
 - a) Pop the top item from stack.
 - b) Print the popped item, set current = popped_item->right
 - c) Go to step 3.
- 5) If current is NULL and stack is empty then we are done.



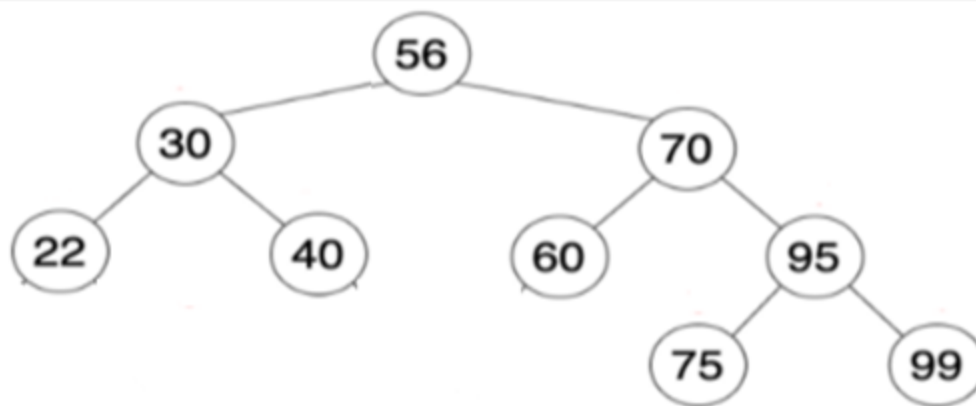
Preorder Binary Search Tree

- 1) Create an empty stack nodeStack and push root node to stack.
- 2) Do the following while nodeStack is not empty.
 -a) Pop an item from the stack and print it.
 -b) Push right child of a popped item to stack
 -c) Push left child of a popped item to stack



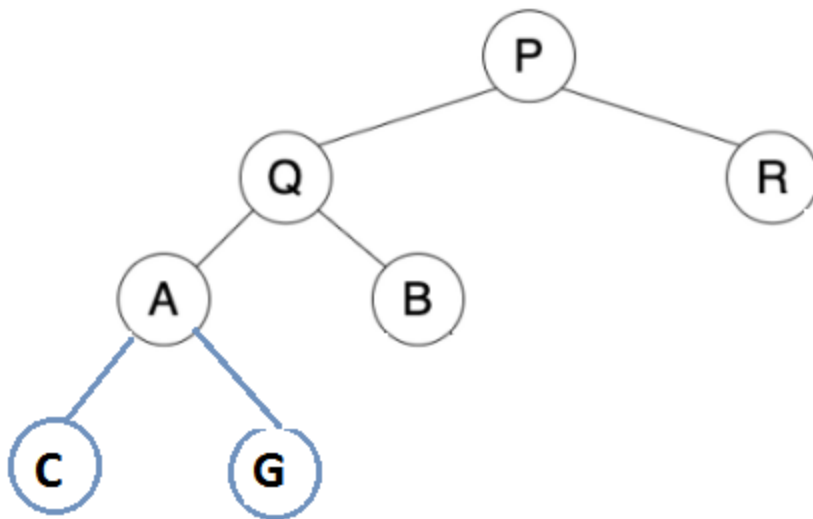
Preorder Binary Search Tree

- 1. Push root to first stack.
- 2. Loop while first stack is not empty
 - 2.1 Pop a node from first stack and push it to second stack
 - 2.2 Push left and right children of the popped node to first stack
- 3. Print contents of second stack



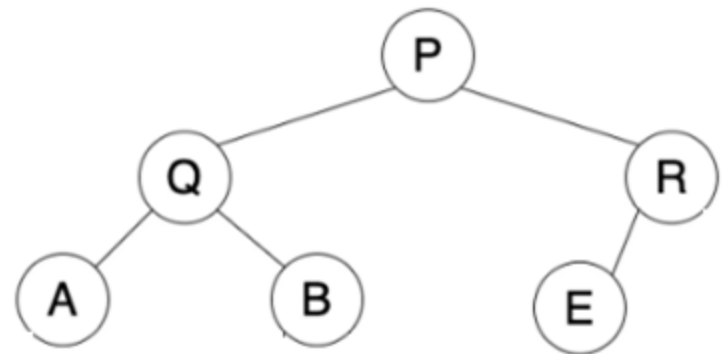
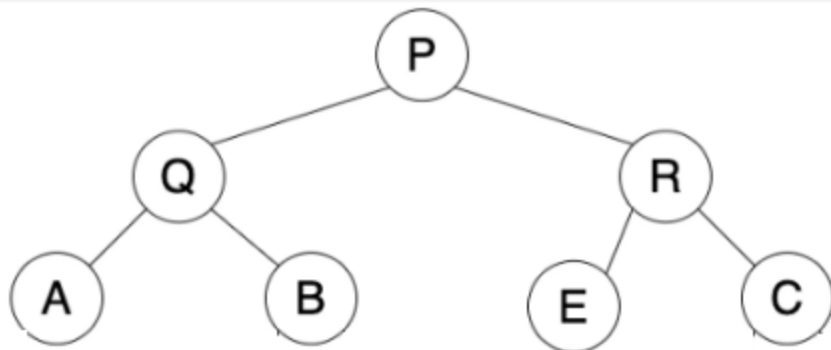
1. Full Binary Tree.

It is a special kind of a binary tree that has either zero children or two children. It means that all the nodes in that binary tree should either have two child nodes of its parent node or the parent node is itself the leaf node or the external node.



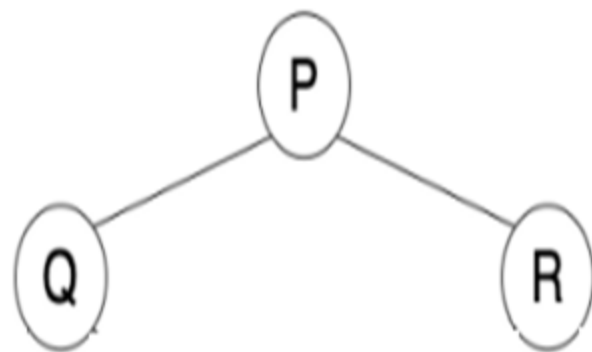
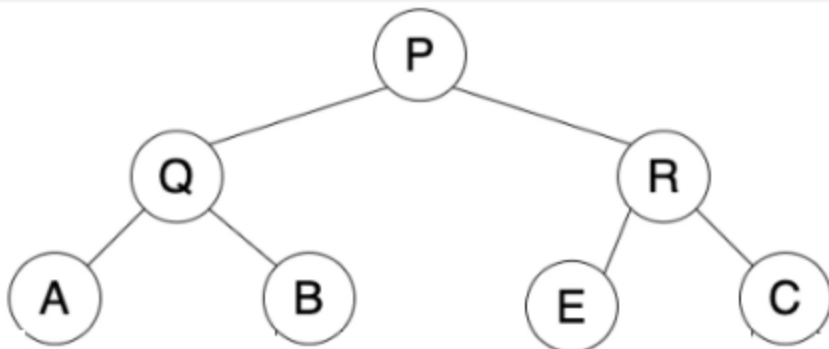
2. Complete Binary Tree.

A complete binary tree is another specific type of binary tree where all the tree levels are filled entirely with nodes, except the lowest level of the tree. Also, in the last or the lowest level of this binary tree, every node should possibly reside on the left side. Here is the structure of a complete binary tree:



3. Perfect Binary Tree.

A binary tree is said to be 'perfect' if all the internal nodes have strictly two children, and every external or leaf node is at the same level or same depth within a tree. A perfect binary tree having height 'h' has $2^h - 1$ nodes. Here is the structure of a perfect binary tree:



Binary Tree Representation

A node of a binary tree is represented by a structure containing a data part and two pointers to other structures of the same type.

